Changelog

All notable changes to the Apollo workspace are documented in this file.
Versioning follows Semantic Versioning 2.0.0.
Change-class tags: [patch] backward-compatible fix, [minor] additive non-breaking, [major] breaking, [arch] cross-cutting redesign.

[Unreleased]

Added

- `apollo-fft` / `application/execution/kernel/winograd.rs` (new module):
  - Algebraic Winograd short-DFT kernels for sizes 2, 4, 8, 16, 32, and 64 (f64 and f32).
  - DFT-2: 0 multiplications (pure add/subtract butterfly).
  - DFT-4: 0 multiplications (±i rotation via swap-and-negate, no trig).
  - DFT-8: 4 real multiplications using W_8^k exact algebraic twiddles (W_8^1 = SQ2O2·(1−i), W_8^2 = −i, W_8^3 = −SQ2O2·(1+i)).
  - DFT-16: 2×DFT-8 + 8 exact nested-square-root 16th-root twiddles.
  - DFT-32: 2×DFT-16 + 16 trigonometric 32nd-root twiddles.
  - DFT-64: 2×DFT-32 + 32 trigonometric 64th-root twiddles.
  - `apply_twiddle_64` / `apply_twiddle_32` helpers for complex multiply.
  - 23 unit tests covering forward, inverse, roundtrip, and boundary cases for all sizes.

Changed

- `apollo-fft` / `application/execution/kernel/mod.rs`:
  - Registered `pub mod winograd;` in the kernel module tree.
- `apollo-fft` / `application/execution/kernel/radix8.rs`:
  - Completely rewritten as a true Winograd-radix-8 DIT FFT replacing the former radix-2 delegate.
  - Implements base-8 digit-reverse permutation (`digit_reverse_64/32`) and an iterative DIT loop
    that calls `winograd::dft8_64/32` as the inner butterfly.
- `apollo-fft` / `application/execution/kernel/radix16.rs`:
  - Replaced the $O(R^2)$ DFT-matrix inner butterfly with `winograd::dft16_64/32`.
  - Removed dead code: `cmul_64`, `cmul_32`, `dft_matrix_64`, `dft_matrix_32`, `radix_r_inplace_64`, `radix_r_inplace_32`.
  - Inner butterfly now costs ~8 real multiplications (Winograd DFT-16) vs. $O(R^2)$ previously.
- `apollo-fft` / `application/execution/kernel/radix32.rs`:
  - Replaced the $O(R^2)$ DFT-matrix inner butterfly with `winograd::dft32_64/32`.
  - Removed same dead code categories as radix16.
- `apollo-fft` / `application/execution/kernel/radix64.rs`:
  - Replaced the $O(R^2)$ DFT-matrix inner butterfly with `winograd::dft64_64/32`.
  - Removed same dead code categories as radix16.

Verification

- `cargo test -p apollo-fft`: **101/101 tests pass** (includes all new radix-8 tests and existing radix-16/32/64 tests).
- `cargo test -p apollo-fft -- winograd`: 23/23 Winograd unit tests pass.
- `cargo run -p apollo-validation --release`: all 59 published-reference fixtures pass; roundtrip max-abs-error ≤ 2.2e-16 (f64), RustFFT delta = 0.
- `cargo bench -p apollo-fft --bench kernel_strategy` (selected results vs previous baseline):
  - `radix16_inplace/16`: 227 ns (−27%, **improved**)
  - `radix64_inplace/64`: 1.22 µs (within noise, stable)
  - `auto_selector/64`: 1.02 µs (−11%, **improved** — auto-selector now routes to faster kernel)
  - `radix32_inplace/32`: 505 ns (+6%, within noise for single-stage transform at N=32)

Previous [Unreleased] — twiddle-table stack-array optimisation (commit c85b301)

Changed

- `apollo-fft` / `application/execution/kernel/radix16.rs`:
  - Removed per-call heap allocation in true radix-16 kernels by replacing dynamic scratch/output
    vectors with fixed-size stack arrays.
  - Reduced twiddle overhead by consuming stage twiddles from caller-provided tables and using
    iterative twiddle recurrence inside butterfly loops.
  - Precomputed radix-16 intra-butterfly DFT matrix once per call to eliminate repeated trigonometric evaluation in inner loops.
- `apollo-fft` / `application/execution/kernel/radix32.rs`:
  - Applied the same zero-extra-allocation stack scratch strategy, twiddle-table stage reuse, and precomputed DFT-matrix butterfly optimization for true radix-32 execution.
- `apollo-fft` / `application/execution/kernel/radix64.rs`:
  - Applied the same zero-extra-allocation stack scratch strategy, twiddle-table stage reuse, and precomputed DFT-matrix butterfly optimization for true radix-64 execution.

Verification

- `cargo test -p apollo-fft radix16 -- --nocapture`: passed.
- `cargo test -p apollo-fft radix32 -- --nocapture`: passed.
- `cargo test -p apollo-fft radix64 -- --nocapture`: passed.
- `cargo bench -p apollo-fft --bench kernel_strategy`: completed; high-radix kernels improved
  materially from previous baselines (for example, `radix64_inplace/64` improved by ~34%).
- `cargo run -p apollo-validation --release`: passed, including external rustfft/numpy checks.
- `D:/miniforge3/python.exe D:/apollofft/crates/apollo-python/tests/benchmark_vs_numpy.py`:
  all Apollo-vs-NumPy output comparisons passed.

[0.13.27] — Closure LXVI

Closure LXVI — apollo-fft: add f32 Stage-4 radix-2 kernel specialization [patch]

Changed

- `apollo-fft` / `application/execution/kernel/radix2.rs`:
  - Added explicit Stage-4 (`len=16`) forward f32 constant-kernel specialization in `forward_inplace_32_with_twiddles`.
  - Added explicit Stage-4 (`len=16`) inverse f32 constant-kernel specialization in `inverse_inplace_unnorm_32_with_twiddles`.
  - Added explicit Stage-4 (`len=16`) inverse f32 constant-kernel specialization in `inverse_inplace_32_with_twiddles` with the existing normalized final-stage fusion preserved.
  - Shifted f32 general-stage loop start from `len=16, base=7` to `len=32, base=15` where Stage-4 specialization applies.

Verification

- `cargo test --workspace --release`: all workspace unit tests and doc tests passed.
- `D:/miniforge3/python.exe tests/benchmark_vs_numpy.py` (from `crates/apollo-python`): all Apollo-vs-NumPy output comparisons passed.

[0.13.26] — Closure LXV

Closure LXV — true radix kernels and initial GPU radix-4 execution path [minor]

Changed

- `apollo-fft`:
  - Implemented true Cooley-Tukey radix kernels for `radix4`, `radix16`, `radix32`, and `radix64`.
  - Kept higher-radix kernels available as explicit APIs while preserving production routing safety.
- `apollo-fft-wgpu`:
  - Added GPU radix-4 execution metadata, planner/pipeline integration, dispatch handling, and WGSL radix-4 shader entry points.

Verification

- `cargo test -p apollo-fft --release`: passed.
- `cargo test -p apollo-fft-wgpu --release`: passed.
- `cargo bench -p apollo-fft --bench kernel_strategy`: post-change measurements collected.
- Python Apollo-vs-NumPy parity benchmark: all output comparisons passed.

[0.13.25] — Closure LXIV

Closure LXIV — apollo-dht: reuse multidimensional lane buffers per plan [patch]

Changed

- `apollo-dht` / `application/execution/plan/dht.rs`: `DhtPlan` now owns reusable lane scratch buffers (`lane_in`/`lane_out`) behind a mutex.
- 2D/3D separable DHT execution paths now reuse plan-owned lane buffers instead of allocating per-call lane vectors for each transform invocation.

Memory and performance impact

- Eliminates repeated `Vec<f64>` lane allocations on `forward_2d`/`inverse_2d` and `forward_3d`/`inverse_3d` call paths.
- Preserves the existing output contracts while reducing allocator traffic for repeated multidimensional transforms.

Verification

- `cargo test -p apollo-dht`: 23 passed, 0 failed.
- Existing output comparison checks remain green for DHT-vs-DFT parity, fast-vs-direct Hartley parity, typed-path checks, and `_into` multidimensional API equivalence.

[0.13.24] — Closure LXIII

Closure LXIII — apollo-dht: remove per-call typed conversion allocations [patch]

Changed

- `apollo-dht` / `application/execution/plan/dht.rs`: `DhtPlan` now owns reusable typed-conversion
  scratch buffers (`input`/`output` f64 vectors) behind a mutex.
- Typed `HartleyStorage` default paths (`f32` and mixed `f16`) now reuse plan-owned scratch buffers instead of allocating fresh conversion vectors on each `forward_typed_into`/`inverse_typed_into` call.

Verification

- Extended typed-path value checks to include `f32` inverse output comparison against the `f64` owner-path inverse reference.
- `cargo test -p apollo-dht`: 23 passed, 0 failed.
- Existing output comparison checks remain green for DHT-vs-DFT parity, fast-vs-direct Hartley parity, and multidimensional `_into` API equivalence.

[0.13.23] — Closure LXII

Closure LXII — apollo-dht: remove remaining 3D separable temporary cube [patch]

Changed

- `apollo-dht` / `application/execution/plan/dht.rs`: `forward_3d_impl` no longer allocates any full `Array3<f64>` temporary for axis staging.
- Axis-1 and axis-2 passes now read each line into reusable lane buffers and write transformed values back to the same line in the destination cube.

Memory and performance impact

- Eliminates the final full `N×N×N` temporary allocation in the 3D separable DHT path.
- Keeps fixed-size lane buffers only, reducing peak memory footprint and copy pressure.

Verification

- `cargo test -p apollo-dht`: 23 passed, 0 failed.
- Output comparison checks remain green for DHT-vs-DFT parity, fast-vs-direct Hartley parity, and `_into` multidimensional APIs vs allocating variants.

[0.13.22] — Closure LXI

Closure LXI — apollo-dht: reduce multidimensional separable temporary storage [patch]

Changed

- `apollo-dht` / `application/execution/plan/dht.rs`: `forward_2d_impl` now uses the caller-provided
  output array as the row-pass workspace, eliminating the dedicated full `Array2<f64>` temporary.
- `apollo-dht` / `application/execution/plan/dht.rs`: `forward_3d_impl` now uses the caller-provided
  output cube for the axis-0 pass and retains only one intermediate `Array3<f64>` for the axis-1 pass, reducing full-cube temporary storage from two allocations to one.

Verification

- Added explicit output-equality coverage for `forward_2d_into`, `inverse_2d_into`, `forward_3d_into`,
  and `inverse_3d_into` against the corresponding allocating APIs.
- `cargo test -p apollo-dht`: 23 passed, 0 failed.

[0.13.21] — Closure LX

Closure LX — apollo-dht: remove redundant fast-wrapper scratch initialization [patch]

Changed

- `apollo-dht` / `infrastructure/kernel/fast.rs`: `dht_fast` now allocates an uninitialized-value-equivalent
  zeroed complex scratch buffer once and delegates population to `dht_fast_with_scratch`, removing the
  previous redundant pass that built `Complex64 { re: x, im: 0.0 }` for every sample before rewriting every slot.

Verification

- Added explicit wrapper-level output comparison coverage: the standalone `dht_fast` path is now checked
  directly against the independent $O(N^2)$ Hartley kernel at `N=512`.
- `cargo test -p apollo-dht`: 21 passed, 0 failed.

[0.13.20] — Closure LIX

Closure LIX — apollo-dht: remove full-result allocation from caller-owned 2D/3D APIs [patch]

Changed

- `apollo-dht` / `application/execution/plan/dht.rs`: added private direct-writer helpers for separable 2D and 3D DHT execution.
- `forward_2d_into` and `forward_3d_into` now execute directly into the caller-owned output arrays instead of allocating a full result array and copying it afterward.
- `inverse_2d_into` and `inverse_3d_into` now reuse the same direct-writer path and apply normalization in-place on the caller-owned output buffer.
- Allocating variants (`forward_2d`, `inverse_2d`, `forward_3d`, `inverse_3d`) now build the result buffer once and dispatch through the same writer helpers, keeping a single authoritative implementation.

Memory and performance impact

- Eliminates one full `Array2<f64>` allocation + copy in `forward_2d_into` and `inverse_2d_into`.
- Eliminates one full `Array3<f64>` allocation + copy in `forward_3d_into` and `inverse_3d_into`.
- Reduces peak memory footprint for caller-owned multidimensional DHT execution while preserving the same lane-buffer reuse and fast-kernel scratch reuse from the previous closures.

Verification

- `cargo test -p apollo-dht`: 20 passed, 0 failed.
- Existing output comparison coverage remains green, including direct Hartley parity at the fast threshold and 2D/3D inverse roundtrip recovery.

[0.13.19] — Closure LVIII

Closure LVIII — apollo-dht: reuse 2D/3D lane buffers in separable plan passes [patch]

Changed

- `apollo-dht` / `application/execution/plan/dht.rs`: removed per-lane `Vec<f64>` allocation in `forward_2d` and `forward_3d`.
- 2D row/column passes now reuse two plan-local vectors per call:
  - `lane_in` for gathering the current row or column
  - `lane_out` for the transform result
- 3D axis-0/1/2 passes now do the same for every fiber instead of allocating a fresh `Vec<f64>` inside each nested loop.

Memory and performance impact

- Eliminates one heap allocation per 2D row and per 2D column transform.
- Eliminates one heap allocation per 3D fiber transform on all three axes.
- Combined with Closure LVII scratch reuse, large separable DHT passes now avoid both:
  - repeated lane-buffer allocation at the plan layer
  - repeated complex FFT scratch allocation in the fast 1D kernel path

Verification

- `cargo test -p apollo-dht`: 20 passed, 0 failed.
- Existing output-comparison coverage remains green, including:
  - direct Hartley parity at the fast threshold
  - 2D separability known-value verification
  - 2D and 3D inverse roundtrip recovery

[0.13.18] — Closure LVII

Closure LVII — apollo-dht: reuse FFT scratch in fast Hartley plan path [patch]

Changed

- `apollo-dht` / `application/execution/plan/dht.rs`: `DhtPlan` now owns an optional reusable `Mutex<Vec<Complex64>>` scratch buffer for lengths at or above `FAST_KERNEL_THRESHOLD`.
- `apollo-dht` / `infrastructure/kernel/fast.rs`: added `dht_fast_with_scratch`, a caller-owned-scratch entry point for the FFT-mapped Hartley kernel.
- `DhtPlan::forward_into` now locks and reuses plan-owned scratch for the fast path instead of allocating a fresh complex buffer on every call.

Memory and performance impact

- Eliminates one `Vec<Complex64>` allocation per fast 1D DHT call.
- This also removes repeated fast-kernel scratch allocation inside 2D and 3D separable DHT passes, because those paths route every row/column/fiber transform through the same

`DhtPlan::forward_into` entry point.

Verification

- Added explicit fast-path parity test at the threshold: the FFT-mapped DHT at `N=512` matches the independent direct Hartley kernel within `1e-10`.
- `cargo test -p apollo-dht`: 20 passed, 0 failed.
- Workspace Python output-comparison benchmark remains clean: 46/46 Apollo-vs-NumPy FFT output comparisons PASS.

[0.13.17] — Closure LVI

Closure LVI — apollo-fft: reduce mixed-radix/radix-4 twiddle overhead with iterative recurrence [patch]

Changed

- `apollo-fft` / `mixed_radix.rs`: removed per-bin `sin`/`cos` twiddle generation in the recursive radix-2 combine stage. Each combine level now computes one complex step root `exp(-2πi/N)` and advances twiddles by repeated complex multiplication.
- `apollo-fft` / `radix4.rs`: removed per-bin `sin`/`cos` twiddle generation in the radix-4 combine stage. Twiddles `W_N^k`, `W_N^{2k}`, and `W_N^{3k}` are now advanced iteratively from a single stage root and its powers.
- The temporary ad hoc comparison example used during profiling was removed after validation to keep the tree clean.

Performance

- Standalone complex-kernel comparison in release mode improved substantially after recurrence-based twiddle stepping:
  - N=16384 mixed-radix vs radix-2: `4.43x` slower -> `3.23x` slower
  - N=65536 mixed-radix vs radix-2: `4.03x` slower -> `2.73x` slower
- Result: the latest kernels are materially cheaper than the initial implementation, but still not competitive with the tuned iterative radix-2 production path, so dispatch remains unchanged.

Verification

- `cargo test -p apollo-fft`: 67 passed, 0 failed.
- Python benchmark output validation: 46/46 Apollo-vs-NumPy comparisons PASS.

[0.13.16] — Closure LV

Closure LV — apollo-fft: add explicit radix-4 and mixed radix-2/radix-4 kernels with validation coverage [minor]

Added

- New kernel module `application/execution/kernel/radix4.rs`:
  - `forward_inplace_64`, `inverse_inplace_unnorm_64`, `inverse_inplace_64`
  - `forward_inplace_32`, `inverse_inplace_unnorm_32`, `inverse_inplace_32`
  - Recursive radix-4 decomposition for power-of-four lengths.
- New kernel module `application/execution/kernel/mixed_radix.rs`:
  - `forward_inplace_64`, `inverse_inplace_unnorm_64`, `inverse_inplace_64`
  - `forward_inplace_32`, `inverse_inplace_unnorm_32`, `inverse_inplace_32`
  - Mixed radix-2/radix-4 recursion for power-of-two lengths with Bluestein fallback for non-power-of-two lengths.
- `kernel/mod.rs` exports the new modules (`radix4`, `mixed_radix`).

Verification

- `cargo test -p apollo-fft`: 67 passed, 0 failed.
- Added kernel tests:
  - `radix4_forward_n16_matches_direct`
  - `radix4_inverse_unnorm_n16_matches_direct`
  - `mixed_forward_n32_matches_direct`
  - `mixed_inverse_unnorm_n32_matches_direct`
- Python output-comparison benchmark remains passing (46/46 output checks) with no regression in published API output correctness.

[0.13.15] — Closure LIV

Closure LIV — apollo-fft: remove extra inverse normalization pass; extend benchmark output comparisons to inverse paths [patch]

Changed

- `apollo-fft` / `radix2.rs` — `inverse_inplace_64` now dispatches directly to `inverse_inplace_64_with_twiddles` (fused-final-stage normalization) instead of calling `inverse_inplace_unnorm_64` followed by a separate O(N) scale loop. This removes one full memory read/write pass over the output buffer.
- `apollo-fft` / `radix2.rs` — `inverse_inplace_32` now dispatches directly to `inverse_inplace_32_with_twiddles` (fused-final-stage normalization) instead of `inverse_inplace_unnorm_32` + extra O(N) scale pass.
- `benchmark_vs_numpy.py` — output validation section now covers inverse APIs in addition to forward APIs:
  - `ifft1` vs `numpy.fft.ifft(...).real`
  - `ifft2` vs `numpy.fft.ifft2(...).real`
  - `ifft3` vs `numpy.fft.ifftn(...).real`
  - `ifft_complex1` vs `numpy.fft.ifft(...)`
    This raises correctness gating from 23 checks to 46 checks before timing output.

Memory and performance impact

- Eliminates one post-kernel normalization sweep in public inverse wrappers (`inverse_inplace_64`, `inverse_inplace_32`), reducing memory bandwidth demand and cache pressure for all inverse callers using those APIs.
- Benchmark run after change shows no forward regressions and restores `fft_complex1` N=16384 to >1× in the observed run (1.04×), indicating the prior sub-1× observation was run-to-run variance rather than a kernel-path regression.

Verification

- 63/63 `cargo test -p apollo-fft` unit tests pass.
- 34/34 Python smoke tests pass.
- Output validation: 46/46 checks pass (forward + inverse, real + complex, 1D/2D/3D).

[0.13.14] — Closure LIII

Closure LIII — apollo-fft: halve post-twiddle reads in real FFT pack/unpack; benchmark output validation [patch]

Changed

- `apollo-fft` / `radix2.rs` — `forward_real_inplace_64`: replaced two post-twiddle reads per pair with one. Proof: `post_twiddles[m-l] = exp(-2πi·(N/2-l)/N) = -conj(post_twiddles[l])`.
  The conjugate pair index twiddle is derived algebraically: `xml = a2 - wl.conj() * b2`.
  The middle element at index `m/2` is simplified analytically: `post_twiddles[m/2] = -i` reduces the unpack formula to `xmid = conj(zmid)` — no twiddle table access or complex multiply. Cache pressure on the post-twiddle table is halved (N=65536: saves 256 KB of reads in the forward unpack loop).
- `apollo-fft` / `radix2.rs` — `inverse_real_inplace_64`: replaced the sequential k=1..m-1 preprocess loop with a pairwise k=1..m/2 loop processing (k, m-k) together from one twiddle read. Derived: `i·conj(wmk) = (wk.im, -wk.re)` where `wmk = -conj(wk)`. The Nyquist bin (k=m/2, self-paired) reduces to `scratch[m/2] = input[m/2].conj()` with no twiddle read. Post-twiddle reads for the inverse preprocess are halved (N=65536: saves 256 KB).
- `benchmark_vs_numpy.py`: added Section 1 (Output Validation) before the timing section. Computes Apollo and NumPy outputs for all 23 tested configurations (1D/2D/3D real, 1D complex, sizes 64–65536) and asserts max absolute error < 1e-9. The script aborts with exit code 1 if any comparison fails, guaranteeing correctness before reporting speedup ratios. The maximum observed errors follow the expected O(sqrt(N)·eps_machine) pattern (e.g. N=65536: 7.1e-13).

Mathematical justification

- `post_twiddles[m-l] = exp(-2πi·(m-l)/N)` with `m = N/2`:
  `= exp(-πi)·exp(2πi·l/N) = -exp(2πi·l/N) = -(exp(-2πi·l/N))* = -conj(post_twiddles[l])`. QED.

- For `k=m/2` (Nyquist): `post_twiddles[m/2] = exp(-πi/2) = -i`; `i·conj(-i) = i·i = i² = -1`; `scratch = (xk + xk* + (-1)(xk - xk*))·0.5 = xk*`. QED.
- For forward middle: same `-i` twiddle yields `xmid = zmid.re - i·zmid.im = conj(zmid)`. QED.

Verification

- 63/63 `cargo test -p apollo-fft` unit tests pass.
- 34/34 Python smoke tests (`test_smoke.py`) pass.
- Output validation: 23/23 configurations PASS with max absolute error < 1e-9 vs NumPy. Worst case: fft3 N=128^3, max_abs = 5.1e-12 (≪ 1e-9 threshold).

[0.13.13] — Closure LII

Closure LII — apollo-fft: cache-sequential gather/scatter for 3D axis-1 and axis-0 passes [patch]

Changed

- `apollo-fft` / `dimension_3d.rs`: reorganized the tile-blocked gather and scatter loops in `axis1_pass_complex` (f64), `axis0_pass_complex` (f64), `axis1_pass_complex_f32`, and `axis0_pass_complex_f32`. The previous code ordered tiles with k (or i) as the innermost loop, producing strided loads from the source array with stride proportional to `ny` or `ny*nz`. The new code promotes i (or j) to the outermost position and makes k the innermost, so reads from `data_slice` are sequential (stride 1) and writes to `scratch` carry the stride. Hardware write-combining buffers the non-sequential stores without pipeline stalls; sequential loads are critical for prefetch and cache-line utilization. This change does not alter the mathematical result or the scratch layout.
- `dimension_3d.rs`: added `src_base = i * ny * nz` and `dst_base` precomputed index variables in the axis-0 gather/scatter to hoist the multiply of `i` out of the inner tile loops, eliminating one address multiplication per (j,k) iteration.
- `dimension_2d.rs`: gather/scatter loop order unchanged (2D matrices fit in L2; the col-outer order avoids write-allocation pressure on the smaller scratch arrays there).

Performance

- 3D 32^3 real FFT: 1.62× → ~2.2× vs NumPy (confirmed across multiple runs, +36%)
- 3D 128^3 real FFT: 1.18× → ~1.22× vs NumPy
- 2D benchmarks: no regression (within run-to-run variance)
- 1D benchmarks: unaffected (no gather/scatter path)

[0.13.12] — Closure LI

Closure LI — apollo-fft: stage-3 (len=8) butterfly specialization, compile-time W_8^j constants [patch]

Changed

- `apollo-fft` / `radix2.rs`: peeled the len=8 butterfly stage out of the general twiddle loop in all six precomputed-twiddle functions (f64/f32 forward, unnormalized inverse, normalized inverse). Stage-3 twiddles W_8^1=(C,∓C), W_8^2=∓i, W_8^3=(-C,∓C) with C=1/√2 are replaced by compile-time `std::f{64,32}::consts::FRAC_1_SQRT_2` constants, eliminating all four twiddle-table reads per chunk and reducing multiplications from 12 to 4 per chunk (j=1 and j=3 each require 2 muls; j=0 bypass and j=2 ∓i have zero muls).
- `apollo-fft` / `radix2.rs`: added `n == 8` early-return paths to normalized inverse functions with fused 1/N scale — no twiddle table access, no final-stage allocations.
- General twiddle loop now starts at `len=16, base=7` in all six functions (stages 1-3 consume 1+2+4=7 twiddle entries). Stages 1, 2, and 3 are now entirely handled by multiply-free bypasses and compile-time constants.
- Combined savings across stages 1-3: N/2 (stage-1) + N/4 (stage-2) + N (stage-3) = 7N/4 multiplications eliminated per forward transform; same for inverse.

Benchmark (v0.13.12 vs v0.13.11, median over 20 trials)

Size	Before	After	Δ
1D real N=64	8.00×	4.83×	var
1D cpx N=64	7.00×	7.83×	+12%
1D cpx N=16384	0.92×	**1.06×**	sub-1× resolved
1D cpx N=65536	1.34×	1.53×	+14%
2D 128×128	1.11×	**1.28×**	+15%
2D 512×512	1.87×	2.08×	+11%
3D 128^3	1.15×	**1.47×**	+28%

[0.13.11] — Closure L

Closure L — apollo-fft: stage-2 (len=4) butterfly specialization, multiply-free W_4^1=±i rotation [patch]

Changed

- `apollo-fft` / `radix2.rs`: peeled the len=4 butterfly stage out of the general twiddle loop in all six precomputed-twiddle functions (f64/f32 forward, unnormalized inverse, normalized inverse). The stage-2 twiddle `W_4^1 = exp(∓2πi/4) = ∓i` reduces to a 90° register swap with sign flip — zero complex multiplications. Together with the stage-1 bypass (Closure XLVII) and the j=0 bypass (Closure XLVIII), stages 1 and 2 are now entirely multiplication-free, saving N/2 multiply-pairs per forward transform and N/2 per inverse.
- `apollo-fft` / `radix2.rs`: added `n == 4` early-return path to normalized inverse (`inverse_inplace_64_with_twiddles`, `inverse_inplace_32_with_twiddles`) that fuses the final
stage scale into the stage-2 rotate+bypass without touching the twiddle table.

- General twiddle loop now starts at `len=8, base=3` in all six functions; the twiddle table layout is unchanged — `base=3` correctly skips the 1 stage-1 entry and 2 stage-2 entries.

Benchmark (v0.13.11 vs v0.13.10, median over 20 trials)

Size	Real speedup vs numpy	Complex speedup vs numpy
1D N=64	8.00× (+38%)	7.00× (+17%)
1D N=256	4.44× (−7%)	4.00× (+23%)
1D N=1024	3.00× (=)	2.00× (+3%)
1D N=4096	2.30× (+6%)	1.31× (+3%)
2D 128×128	1.11× (+1%)	—

[0.13.10] — Closure XLIX

Closure XLIX — apollo-fft/apollo-python: scalar butterfly hot loop + single-copy complex Python wrappers [patch]

Changed

- `apollo-fft` / `radix2.rs`: replaced operator-based hot-loop complex arithmetic (`t = w * v; u ± t`) with explicit scalar real/imag fused arithmetic in all precomputed-twiddle butterflies (f64/f32 forward + inverse variants, including normalized final stages).
  This removes temporary `Complex` values in the inner loop and improves autovectorization opportunities under `target-cpu=native`.
- `apollo-fft` / `radix2.rs`: restored j=0 bypass in `inverse_inplace_unnorm_64_with_twiddles` (`W_L^0 = 1+0i`), matching the optimized forward and normalized inverse paths and removing one unnecessary complex multiply per chunk per stage.
- `apollo-python` / `lib.rs`: rewired complex FFT wrappers (`fft_complex{1,2,3}`, `ifft_complex{1,2,3}` and plan methods) to use in-place APIs on one owned ndarray buffer instead of the prior double-copy path (`to_owned()` + library-level clone-return API).
  This reduces peak host memory traffic for complex transforms and lowers Python binding overhead.

Verification

- `cargo test -p apollo-fft -p apollo-python` → pass (`apollo-fft`: 63/63).
- `python -m pytest tests/test_smoke.py -q` → pass (34/34).

Benchmark Highlights (vs numpy, Closure XLIX run)

- 1D complex FFT: `N=4096` improved to **1.27x**; `N=65536` improved to **1.37x**.
- 1D real FFT: `N=65536` measured **2.84x**.
- 3D real FFT: `N=128^3` measured **1.57x**.

[0.13.9] — Closure XLVIII

Closure XLVIII — apollo-fft: codegen-units=1, thin LTO, target-cpu=native, j=0 butterfly bypass [patch]

Changed

- `Cargo.toml` workspace `[profile.release]`: set `codegen-units = 1` (single LLVM module for the entire workspace) and `lto = "thin"` (cross-crate inlining without fat-LTO compile time overhead). Together these allow LLVM to inline `num_complex::Complex64::mul` directly into the butterfly loops and apply AVX2/FMA autovectorization across crate boundaries.
- `.cargo/config.toml [build] rustflags`: added `["-C", "target-cpu=native"]` to enable all CPU features on the build machine (AVX2, FMA, BMI2). Butterfly arithmetic now uses `vfmadd256` where applicable.
- `apollo-fft` / `radix2.rs`: j=0 bypass applied to all six butterfly inner loops (`forward_inplace_64`, `inverse_inplace_unnorm_64`, `inverse_inplace_64` intermediate and final stages, and their f32 counterparts). W_L^0 = exp(0) = 1+0i for every stage, so `t = twiddles[0] * hi[0]` reduces to `t = hi[0]`; the j=0 case is hoisted out of the loop as a scalar add/subtract, eliminating one complex multiply per chunk per stage (~3% arithmetic reduction for large N). The inner loop is now `for j in 1..half`, improving loop-bound regularity for LLVM unrolling heuristics.

[0.13.8] — Closure XLVII

Closure XLVII — apollo-fft: O(N) bit-reversal, stage-1 no-mul, split_at_mut butterfly, fused IFFT scale [patch]

Changed

- `apollo-fft` / `radix2.rs`: replaced O(N log N) `bit_reverse()` per-element lookup with the O(N) iterative XOR/binary-counter-in-reverse technique for both `bit_reverse_permutation_64` and `bit_reverse_permutation_32`. The algorithm maintains `j = bit_reverse(i, log_n)` via amortized bit-flip operations (≈2 per element average), replacing the prior $N \cdot \log_2 N$ inner loop.
- `apollo-fft` / `radix2.rs`: stage-1 (len=2) butterfly is now a special case that omits the twiddle multiplication. W_2^0 = 1+0i for all N; `(u + 1·v, u - 1·v) = (u+v, u-v)` eliminates N/2 complex multiplications per forward or inverse transform call.
- `apollo-fft` / `radix2.rs`: all butterfly inner loops use `chunk.split_at_mut(half)` to split each chunk into non-overlapping `lo` and `hi` halves. Exposes non-aliasing to LLVM, enabling autovectorization of the `j`-loop across butterfly pairs.
- `apollo-fft` / `radix2.rs`: `inverse_inplace_64_with_twiddles` and `inverse_inplace_32_with_twiddles` are now inlined butterfly loops. The `1/N` scale is fused into the final butterfly stage, eliminating a separate O(N) normalization pass (one full array read+write removed per IFFT call). Proof: `(u + t) * scale = unnorm_out[k] / N = norm_out[k]`.

Performance (Closure XLVII vs numpy baseline)

Case	After XLVI	After XLVII	Change
1D real N=64	5.80×	**6.40×**	+10%
1D real N=1024	2.42×	**2.69×**	+11%
1D real N=4096	1.62×	**2.03×**	+25%
1D cpx N=64	4.67×	**6.40×**	+37%
1D cpx N=4096	0.93×	**1.06×**	fixed!
1D cpx N=65536	0.91×	**1.16×**	fixed!
2D 32×32	2.01×	**2.54×**	+26%
2D 64×64	1.19×	**1.37×**	+15%
3D 8^3	3.16×	**6.57×**	+108%
3D 128^3	1.08×	**1.23×**	+14%

[0.13.7] — Closure XLVI

Closure XLVI — apollo-fft: iRFFT half-spectrum inverse, cache-blocked 3D gather/scatter [patch]

Changed

- `apollo-fft` / `radix2.rs`: added `inverse_real_inplace_64(input, output, scratch, fft_twiddles, post_twiddles)`.
  Pre-processes the N/2+1-point Hermitian spectrum into M=N/2 complex values via `Z[k] = (X[k]+conj(X[M-k]))/2 + i·conj(W_k)·(X[k]-conj(X[M-k]))/2` (k=1..M-1) using the
  same `post_twiddles` as the forward pass; applies an M-point normalized IFFT via `inverse_inplace_64_with_twiddles`; unpacks `x[2k]=Z[k].re`, `x[2k+1]=Z[k].im`. Work ≈ $N/2 \cdot \log_2(N/2) + O(N)$ vs the previous $N \cdot \log_2 N$ IFFT of the full complex spectrum.
  Normalization verified: `inverse_inplace_64_with_twiddles` on M elements divides by M, matching the 1/N normalization of the full N-point IFFT.
- `apollo-fft` / `dimension_1d.rs`: `FftPlan1D` gains `real_inv_scratch: Option<Mutex<Vec<Complex64>>>`
  (M=N/2 entries, allocated for PoT N ≥ 4). `inverse_complex_to_real_with_workspace` and `inverse_complex_to_real_into` now dispatch to `inverse_real_inplace_64` when all fast-path fields (`twiddle_inv_64`, `real_fwd_post_twiddles`, `real_inv_scratch`) are available.
- `apollo-fft` / `dimension_3d.rs`: added `const GATHER_TILE: usize = 32`. Both `axis1_pass_complex` (f64/f32) and `axis0_pass_complex` (f64/f32) replace plain nested gather and scatter loops with GATHER_TILE×GATHER_TILE-blocked loops. For axis-1 the (j,k)-plane is tiled per i-slice; for axis-0 the (j,k)-plane is tiled with inner i-stride. A 32×32 tile of Complex64 = 16 KB, fitting in L1 cache (32–48 KB), eliminating cold-miss penalties during

non-contiguous axis transposes. Applied to all four gather/scatter sites (f64 axis-1, f64 axis-0, f32 axis-1, f32 axis-0).

Performance (Closure XLVI vs numpy baseline)

Case	After XLV	After XLVI	Change
3D 32^3	0.95×	**1.48×**	+56%
2D 1024×1024	1.06×	**1.84×**	+73%
2D 128×128	0.85×	**0.93×**	+9%
1D real N=256	3.64×	**4.27×**	+17%
1D real N=1024	2.14×	**2.42×**	+13%

[0.13.6] — Closure XLV

Closure XLV — apollo-fft: real FFT half-spectrum trick, rayon sequential threshold, cache-blocked transpose [patch]

Changed

- `apollo-fft` / `radix2.rs`: added `build_real_fwd_post_twiddles_64(n) -> Vec<Complex64>` and `forward_real_inplace_64(input, output, fft_twiddles, post_twiddles)`. The new function packs a real N-input into N/2 complex samples, applies an N/2-point forward FFT using the first N/2-1 entries of the existing N-point twiddle table (contiguous-layout invariant), then unpacks in-place via the split-radix identity `X[k] = (Z[k]+Z[M-k]*)/2 - i·W_N^k·(Z[k]-Z[M-k]*)/2`, processing symmetric pairs to avoid aliasing. Complexity ≈ $N/2 \cdot \log_2(N/2) + O(N)$ vs $N \cdot \log_2 N$ for the previous zero-padded complex FFT path.
- `apollo-fft` / `dimension_1d.rs`: `FftPlan1D` gains a `real_fwd_post_twiddles: Option<Vec<Complex64>>` field (N/2+1 entries, built at construction for PoT N ≥ 4). Both `forward_real_to_complex` and `forward_real_to_complex_into` now dispatch to `forward_real_inplace_64` when post-twiddles are available, falling back to the previous complex-pad path for non-PoT or N < 4.
- `apollo-fft` / `dimension_2d.rs`: added `const RAYON_THRESHOLD: usize = 32768` and `const TRANSPOSE_TILE: usize = 32`. All four axis-pass functions (f64 row/col, f32 row/col) now use sequential `chunks_mut` iteration when `data.len() ≤ RAYON_THRESHOLD`, eliminating rayon task-spawn overhead for small matrices. The column gather and scatter loops are now 32×32-tile cache-blocked, keeping each tile ≤ 8 KB in L1.
- `apollo-fft` / `dimension_3d.rs`: same `RAYON_THRESHOLD = 32768` applied to all six axis-pass functions (axis0/1/2 × f64/f32). Sequential path used for volumes ≤ 32K elements.

Performance (Closure XLV vs numpy baseline)

Case	Before XLV	After XLV	Change
1D real N=1024	1.57×	2.14×	+36%
1D real N=4096	0.95×	1.66×	+75%
1D real N=16384	3.18×	6.16×	+94%
2D 32×32	0.37×	2.03×	+449%
2D 64×64	0.55×	1.13×	+105%
2D 128×128	0.69×	0.85×	+23%
3D 8^3	0.48×	4.27×	+789%
3D 16^3	1.22×	1.79×	+47%
1D cpx N=1024	1.38×	1.24×	-10%

[0.13.5] — Closure XLIV

Closure XLIV — apollo-fft: precomputed twiddle tables + preallocated scratch in 2D/3D plans [patch]

Changed

- `apollo-fft` / `dimension_2d.rs`: `FftPlan2D` now embeds eight per-axis per-direction twiddle table fields (`twiddle_row_fwd_64`, `twiddle_row_inv_64`, `twiddle_col_fwd_64`, `twiddle_col_inv_64`, and f32 variants). Row (axis-1) and column (axis-0) butterfly passes use `forward_inplace_64_with_twiddles` / `inverse_inplace_64_with_twiddles` when the axis length is
  a power of two, eliminating the per-lane `build_forward_twiddle_table_64` allocation that previously occurred on every axis pass. Fallback to `fft_forward_64` for non-power-of-two axes.
- `apollo-fft` / `dimension_2d.rs`: column-pass scratch buffer preallocated at plan construction time (`scratch_col_64`, `scratch_col_32`; each `nx * ny` entries) and reused via `Mutex`, eliminating the per-call `Vec::new(nx * ny)` allocation in `axis0_pass_complex`.
- `apollo-fft` / `dimension_3d.rs`: `FftPlan3D` receives the same treatment — twelve per-axis twiddle table fields + four preallocated scratch buffers (`scratch_y_64`, `scratch_x_64`, `scratch_y_32`, `scratch_x_32`; each `nx * ny * nz` entries). All z/y/x butterfly passes use precomputed tables for power-of-two axis lengths.

Performance

Transform	Before	After	Delta
2D 256×256 (vs numpy)	1.00×	**1.33×**	+33%
2D 512×512 (vs numpy)	1.12×	**1.35×**	+20%
2D 1024×1024 (vs numpy)	1.17×	**1.45×**	+24%

Transform	Before	After	Delta
3D 32^3 (vs numpy)	0.40×	**1.26×**	+216%
3D 64^3 (vs numpy)	0.87×	**1.14×**	+31%
3D 128^3 (vs numpy)	1.04×	**1.20×**	+15%

[0.13.4] — Closure XLIII

Closure XLIII — apollo-fft: contiguous per-stage twiddle tables; eliminate per-call allocation [patch]

Changed

- `apollo-fft` / `radix2.rs`: replaced unified N/2 strided twiddle table with contiguous per-stage layout. Stage s (group length `2^s`) occupies `2^(s-1)` sequential entries; the butterfly inner loop reads `stage_twiddles[j]` with no stride, eliminating L1 cache misses at N ≥ 256.
- `apollo-fft` / `radix2.rs`: `forward_inplace_64`, `inverse_inplace_unnorm_64`, `forward_inplace_32`, `inverse_inplace_unnorm_32` delegate to new `*_with_twiddles` kernels; four new exported table-building functions (`build_forward_twiddle_table_{32,64}`, `build_inverse_twiddle_table_{32,64}`).
- `apollo-fft` / `dimension_1d.rs`: `FftPlan1D` now precomputes and stores contiguous per-stage twiddle tables for power-of-two N at plan construction time (in `FFT_CACHE_1D`). Per-call Vec allocation eliminated from all hot paths. `forward_complex_slice_inplace`, `inverse_complex_slice_inplace`, `forward_f32`, `inverse_f32` use stored tables.

Performance

Transform	N	Before	After	Delta
1D real (vs numpy)	1024	0.89×	**1.51×**	+70%
1D real (vs numpy)	4096	0.61×	**0.98×**	+61%
1D complex (vs numpy)	1024	(unknown)	**1.16×**	—
1D complex (vs numpy)	4096	0.57×	**1.04×**	+83%

[0.13.3] — Closure XLII

Closure XLII — apollo-python: complete Python bindings; numpy FFT benchmark [minor]

Added

- `apollo-python`: `fft_complex1`, `ifft_complex1` — complex128→complex128 1D FFT, numpy-compatible.
- `apollo-python`: `fft_complex2`, `ifft_complex2` — complex128 2D FFT.
- `apollo-python`: `fft_complex3`, `ifft_complex3` — complex128 3D FFT.

- `apollo-python`: `fftfreq(n, d=1.0)` — numpy-compatible DFT frequency bin centers.
- `apollo-python`: `rfftfreq(n, d=1.0)` — non-negative bins for real-input FFT.
- `apollo-python`: `fftshift(x)` — shift zero-frequency to center.
- `apollo-python`: `ifftshift(x)` — inverse of fftshift.
- `apollo-python`: `dht1`, `idht1` — 1D Discrete Hartley Transform (forward / scaled inverse).
- `apollo-python`: `dht2`, `idht2` — 2D DHT on square N×N arrays.
- `apollo-python`: `dht3`, `idht3` — 3D DHT on cubic N×N×N arrays.
- `apollo-python`: `fwht1`, `ifwht1` — 1D Fast Walsh-Hadamard Transform (N power of two).
- `apollo-python`: `fwht2`, `ifwht2` — 2D FWHT on square N×N arrays.
- `apollo-python`: `fwht3`, `ifwht3` — 3D FWHT on cubic N×N×N arrays.
- `apollo-python`: `dct2_1d`, `idct2_1d` — unnormalized DCT-II and its inverse (DCT-III × 2/N).
- `apollo-python`: `dst2_1d`, `idst2_1d` — unnormalized DST-II and its inverse.
- `apollo-python`: `FftPlan1D.fft_complex` / `FftPlan1D.ifft_complex` — plan-based complex 1D FFT.
- `apollo-python`: `FftPlan2D.fft_complex` / `FftPlan2D.ifft_complex` — plan-based complex 2D FFT.
- `crates/apollo-python/Cargo.toml`: added `apollo-dht`, `apollo-fwht`, `apollo-dctdst` dependencies.
- `tests/benchmark_vs_numpy.py`: empirical 1D/2D/3D Apollo vs numpy.fft performance comparison.
- 19 new smoke tests covering all new Python bindings (34 total, all passing).

Performance highlights (CPU, release build, median of 20 trials, Windows x64)

Transform	Apollo fastest	numpy fastest	Notes
1D FFT (real, N=64)	**3.3× faster**	—	PyO3 call overhead advantage at small N
1D FFT (real, N=16384)	**2.2× faster**	—	Large-N Cooley-Tukey wins
1D FFT (complex, N=64)	**3.9× faster**	—	
2D FFT (N=1024×1024)	**1.31× faster**	—	Parallel separable row/col FFT
3D FFT (N=128^3)	**1.04× faster**	—	Near-parity; CPU overhead dominates
1D FFT (real, N=4096)	—	**1.63× faster**	numpy FFTPACK cache-hot midrange

[0.13.2] — Closure XLI

Closure XLI — DHT CPU 2D/3D; FWHT CPU 2D/3D; FFT fftfreq/rfftfreq/fftshift/ifftshift [minor]

Added

- `apollo-dht`: `DhtPlan::forward_2d`, `forward_2d_into`, `inverse_2d`, `inverse_2d_into`, `forward_3d`, `forward_3d_into`, `inverse_3d`, `inverse_3d_into` — separable N×N and N×N×N CPU DHT.
- `apollo-dht`: `DhtError::ShapeMismatch2d { expected, rows, cols }` and `DhtError::ShapeMismatch3d { expected, d0, d1, d2 }` for non-square/non-cubic rejection.
- `apollo-dht`: re-exports `ndarray::Array2` and `ndarray::Array3` from crate root.
- `apollo-fwht`: `FwhtPlan2D` — separable N×N FWHT plan with `forward`, `forward_into`, `forward_inplace`, `inverse`, `inverse_into`, `inverse_inplace`, `forward_complex`, `inverse_complex`.
- `apollo-fwht`: `FwhtPlan3D` — separable N×N×N FWHT plan with matching API surface.
- `apollo-fwht`: `FwhtPlan2D` and `FwhtPlan3D` re-exported from crate root.
- `apollo-fft`: `fftfreq(n, d) -> Vec<f64>` — numpy-compatible DFT frequency bin centers.
- `apollo-fft`: `rfftfreq(n, d) -> Vec<f64>` — non-negative bins for real-input FFT, length n/2+1.
- `apollo-fft`: `fftshift<T: Copy>(input: &[T]) -> Vec<T>` — zero-frequency centering.
- `apollo-fft`: `ifftshift<T: Copy>(input: &[T]) -> Vec<T>` — inverse of `fftshift`.
- All four FFT utilities re-exported from `apollo-fft` crate root.

Verification

- `cargo test -p apollo-dht`: 19 passed, 0 failed.
- `cargo test -p apollo-fwht`: 24 passed, 0 failed.
- `cargo test -p apollo-fft`: 63 passed, 0 failed.
- `cargo test -p apollo-validation -- --include-ignored`: 3 passed, 0 failed.

[0.13.1] — Closure XL

Closure XL — GPU DCT/DST 2D and 3D Separable Execution [minor]

Added

- `apollo-dctdst-wgpu` `DctDstWgpuBackend` now exposes separable multidimensional GPU APIs: `execute_forward_2d`, `execute_inverse_2d`, `execute_forward_3d`, `execute_inverse_3d`.
- `WgpuError::ShapeMismatch { expected, rows, cols }` — returned when a 2D input is not `N×N`.
- `WgpuError::ShapeMismatch3d { expected, d0, d1, d2 }` — returned when a 3D input is not `N×N×N`.
- `ndarray = "0.16"` added to `apollo-dctdst-wgpu` dependencies; `Array2` and `Array3` re-exported
  from `apollo-dctdst-wgpu`.
- Verification coverage added in `apollo-dctdst-wgpu`:
  - 2D DCT-II forward GPU parity against CPU separable reference.
  - 2D DCT-II inverse roundtrip recovery.
  - 3D DCT-II forward GPU parity against CPU separable reference.
  - 3D DCT-II inverse roundtrip recovery.
  - Non-square 2D shape rejection (`ShapeMismatch`).

    - Non-cubic 3D shape rejection (`ShapeMismatch3d`).
- Separable strategy: 1D GPU kernel dispatched per row/column/fiber — no new WGSL shaders required.

Final state

- `cargo test -p apollo-dctdst-wgpu`: 28 passed, 0 FAILED, 0 ignored.
- `cargo test -p apollo-validation -- --include-ignored`: 3 passed, 0 FAILED, 0 ignored.
- `apollo-dctdst-wgpu` dimensional parity gap in `gap_audit.md` closed.

[0.13.0] — Closure XXXIX

Closure XXXIX — CPU DCT/DST 2D and 3D Separable Plans [minor]

Added

- `apollo-dctdst` `DctDstPlan` now exposes separable multidimensional CPU APIs: `forward_2d`, `forward_2d_into`, `inverse_2d`, `inverse_2d_into`, `forward_3d`, `forward_3d_into`, `inverse_3d`, and `inverse_3d_into`.
- 2D execution applies the configured real transform kind along rows then columns. 3D execution applies the configured real transform kind along z, then y, then x.
- Shape contracts are explicit and value-checked:
    - 2D methods require square `N x N` arrays for plan length `N`.
    - 3D methods require cubic `N x N x N` arrays for plan length `N`. Mismatches return `DctDstError::LengthMismatch`.
- Verification coverage added in `apollo-dctdst`:
    - 2D separable parity against manual row/column application.
    - 2D inverse roundtrip recovery.
    - 3D inverse roundtrip recovery.
    - Non-square/non-cubic shape rejection.

Final state

- `cargo test -p apollo-dctdst`: 42 passed, 0 FAILED, 0 ignored.
- `apollo-dctdst` README updated with 2D/3D execution surfaces and verification scope.

[0.12.18] — Closure XXXVIII

Closure XXXVIII — DCT-I and DST-I Forward Known-Value Fixtures [patch]

Added

- Validation fixture 58 in `apollo-validation`: `dct1_three_point_forward_known_values_fixture` — DCT-I, N=3, x=[1,2,3]: y=[8,−2,0]; boundary formula y[k]=x[0]+(−1)^k·x[N−1]+2·Σx[n]cos(πnk/(N−1));

y[2]=0 algebraically exact (cos(π)=−1 cancels interior term 4); threshold 1×10^{-15}.
Reference: Rao & Yip (1990) *Discrete Cosine Transform* Table 2.1; FFTW REDFT00.

- Validation fixture 59 in `apollo-validation`: `dst1_two_point_forward_known_values_fixture` —
  DST-I, N=2, x=[1,3]: y=[4√3,−2√3]; formula y[k]=2·Σx[n]sin(π(n+1)(k+1)/(N+1));
  analytically derived as 2·(√3/2+3√3/2)=4√3 and 2·(√3/2−3√3/2)=−2√3; threshold 1×10^{-12}.
  Reference: Rao & Yip (1990) *Discrete Cosine Transform* Table 3.1; FFTW RODFT00.
- Root `README.md` fixture count updated 57 → 59; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 57 → 59.
- All 3 validation tests pass (59 fixtures, 59 attempted, all passed).

[0.12.17] — Closure XXXVII

Closure XXXVII — DCT-III and DST-III Published-Reference Fixtures [patch]

Added

- Validation fixture 56 in `apollo-validation`: `dct3_dc_input_flat_output_fixture` —
  DCT-III, N=4, DC input [1,0,0,0]: y[k]=x[0]/2=1/2 for all k; flat output [½,½,½,½].
  Single-term kernel evaluation (x[n]=0 for n≥1 eliminates all cosine terms);
  Makhoul (1980) IEEE Trans. ASSP 28(1) Table I; FFTW REDFT01; threshold 1×10^{-15}.
- Validation fixture 57 in `apollo-validation`:
  `dst3_nyquist_input_alternating_output_fixture` —
  DST-III, N=4, Nyquist input [0,0,0,1]: y[k]=(−1)^k/2; alternating [½,−½,½,−½].
  Single-term kernel evaluation (x[n]=0 for n≤2 eliminates all sine terms);
  Makhoul (1980) IEEE Trans. ASSP 28(1) Table II; FFTW RODFT01; threshold 1×10^{-15}.
- Root `README.md` fixture count updated 55 → 57; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 55 → 57.
- `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

[0.12.16] — Closure XXXVI

Closure XXXVI — CWT Ricker Impulse Peak and Scale-Normalization Fixtures [patch]

Added

- Validation fixture 54 in `apollo-validation`: `cwt_ricker_impulse_peak_value_fixture` —
  CWT Ricker, N=7, impulse at n_0=3, a=1: W(1,2)=0, W(1,3)=ψ(0)=2/(√3·π^¼), W(1,4)=0.
  W(1,3) is single-tap (no summation error); W(1,2) and W(1,4) are exact zeros
  because $(1-(\pm1)^2)=0$; Daubechies (1992) §2.1 eq.(2.1.4); threshold 1×10^{-14}.

- Validation fixture 55 in `apollo-validation`: `cwt_ricker_scale_normalization_fixture` — CWT Ricker, N=7, impulse at n_0=3, a=2: W(2,3)=ψ(0)/√2=√2/(√3·π^¼).
  Tests the 1/√a L^2 normalization convention directly;
  Daubechies (1992) §2.1; Grossmann & Morlet (1984) eq.(1.3); threshold 1×10^{-13}.
- Root `README.md` fixture count updated 53 → 55; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 53 → 55.
- `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

[0.12.15] — Closure XXXV

Closure XXXV — Daubechies-4 DWT Coefficient and Reconstruction Fixtures [patch]

Added

- Validation fixture 52 in `apollo-validation`: `wavelet_daubechies4_one_level_known_coefficients_fixture` — DWT db4, N=4, level=1, x=[1,0,0,0]: [a0,a1,d0,d1]=[h0,h2,h3,h1] with Daubechies taps h=[0.4829629131, 0.8365163037, 0.2241438680, -0.1294095226].
  Basis-impulse input makes each coefficient a single tap (no summation error);
  threshold 1×10^{-15}.
- Validation fixture 53 in `apollo-validation`: `wavelet_daubechies4_inverse_perfect_reconstruction_fixture` — DWT db4, N=4, level=1: IDWT(DWT([1,-2,0.5,4]))=[1,-2,0.5,4].
  Orthogonal two-channel PR theorem (Mallat 1989, Theorem 2);
  threshold 1×10^{-12}.
- Root `README.md` fixture count updated 51 → 53; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 51 → 53.
- `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

[0.12.14] — Closure XXXIV

Closure XXXIV — CZT Off-Unit-Circle and Hilbert Envelope Fixtures [patch]

Added

- Validation fixture 50 in `apollo-validation`: `czt_off_unit_circle_z_transform_fixture` — CZT N=2, M=2, A=2, W=exp(−πi): X=[1.5+0i, 0.5+0i].
  Evaluates Z-transform off the unit circle at z={2,−2} (|z|=2);
  A=2 factors are dyadic rationals, exact in f64; accumulated FP error=0;
  Rabiner, Schafer & Rader (1969) IEEE TAE 17(2) §II; threshold 1×10^{-12}.

- Validation fixture 51 in `apollo-validation`: `hilbert_pure_cosine_envelope_is_unity_fixture` — Hilbert envelope of x=[1,0,−1,0]=cos(πn/2), N=4: envelope=[1,1,1,1]. DFT mask {0,1,2}×{1,i,−1,−i}; analytic signal=[1,i,−1,−i]; |z[n]|=1 exact; Oppenheim & Schafer (2010) DTSP 3rd ed. §12.1 eq.(12.8); Bedrosian (1963); threshold 1×10^{-12}.
- Root `README.md` fixture count updated 49 → 51; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 49 → 51.
- `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

[0.12.13] — Closure XXXIII

Closure XXXIII — SDFT Sliding Recurrence and FrFT Order-4 Identity Fixtures [patch]

Added

- Validation fixture 48 in `apollo-validation`: `sdft_sliding_recurrence_unit_impulse_all_bins_fixture` — SDFT N=4, zero_state, 4 updates fed [1,0,0,0]: all 4 bins = 1+0i. Tests the sliding-update recurrence path (not direct_bins); factors ∈{1,i,−1,−i}; exact integer result; Jacobsen & Lyons (2003) IEEE SPM 20(2) §2 eq.(2); threshold 1×10^{-12}.
- Validation fixture 49 in `apollo-validation`: `frft_order4_identity_fixture` — UnitaryFrFT N=4, order α=4.0: DFrFT_4([1,2,3,4])=[1,2,3,4]. exp(−4kπi/2)=exp(−2πki)=1; V·I·V^T=I; exact regardless of eigenvector ordering; Candan, Kutay & Ozaktas (2000) IEEE TSP 48(5) §II Corollary; threshold 1×10^{-12}.
- Root `README.md` fixture count updated 47 → 49; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 47 → 49.
- `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

[0.12.12] — Closure XXXII

Closure XXXII — NUFFT Adjoint Identity and Radon Fourier Slice Theorem Fixtures [patch]

Added

- Validation fixture 46 in `apollo-validation`: `nufft_type1_type2_adjoint_inner_product_fixture` — NUFFT N=2, pos=[0,0.5], c=[1,2], f=[3,4]: Re(⟨A·c,f⟩)=Re(⟨c,A*·f⟩)=5.

All exp factors ∈{1,−1}; computation exact in f64; accumulated FP error=0;
Dutt & Rokhlin (1993) SIAM J. Sci. Comput. 14(6): adjoint identity (1.8);
Greengard & Lee (2004) §2; threshold 1×10^{-12}.

- Validation fixture 47 in `apollo-validation`: `radon_fourier_slice_theorem_theta0_fixture` —
  Radon θ=0 Fourier Slice Theorem on 2×2 image [[1,2],[3,4]]:
  DFT_1(R_{θ=0}f)=[10+0i,−2+0i] equals horizontal slice of 2D DFT;
  all DFT factors ∈{1,−1}; exact in f64; Natterer (1986) §I.2 Thm 1.1;
  Radon (1917); threshold 1×10^{-12}.
- Root `README.md` fixture count updated 45 → 47; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 45 → 47.
- `cargo test -p apollo-validation`: 3 passed, 0 FAILED, 0 ignored.

[0.12.11] — Closure XXXI

Closure XXXI — DCT-I and DST-I Self-Inverse Published-Reference Fixtures [patch]

Added

- Validation fixture 44 in `apollo-validation`: `dct1_inverse_roundtrip_three_point_fixture` —
  DCT-I N=3: `IDCT-I(DCT-I([1,2,3])) = [1,2,3]`. Makhoul (1980) IEEE Trans. ASSP 28(1):
  DCT-I self-inverse $C1^2=2(N-1)\cdot I$; FFTW REDFT00: IDCT-I=(1/(2(N−1)))·DCT-I.
  Intermediate spectrum [8,−2,0] is exactly integer (cos values in {−1,0,1});
  round-trip error = 0 analytically; threshold 1×10^{-14}.
- Validation fixture 45 in `apollo-validation`: `dst1_inverse_roundtrip_two_point_fixture` —
  DST-I N=2: `IDST-I(DST-I([1,3])) = [1,3]`. Makhoul (1980) IEEE Trans. ASSP 28(1):
  DST-I self-inverse $S1^2=2(N+1)\cdot I$; FFTW RODFT00: IDST-I=(1/(2(N+1)))·DST-I.
  Intermediate spectrum [4√3,−2√3]; O(ε_f64) error; threshold 1×10^{-14}.
- Root `README.md` fixture count updated 43 → 45; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 43 → 45.
- `cargo test -p apollo-validation -p apollo-dctdst`: 0 FAILED, 0 ignored.

[0.12.10] — Closure XXX

Closure XXX — DCT-IV and DST-IV Self-Inverse Published-Reference Fixtures [patch]

Added

- Validation fixture 42 in `apollo-validation`: `dct4_inverse_roundtrip_two_point_fixture` — DCT-IV N=2: `IDCT-IV(DCT-IV([1,3])) = [1,3]`. Makhoul (1980) IEEE Trans. ASSP 28(1): DCT-IV self-inverse property $C4^2=N\cdot I$; FFTW REDFT11 IDCT-IV=(1/2N)·DCT-IV; threshold 1×10^{-14}.
- Validation fixture 43 in `apollo-validation`: `dst4_inverse_roundtrip_two_point_fixture` — DST-IV N=2: `IDST-IV(DST-IV([2,5])) = [2,5]`. Makhoul (1980) IEEE Trans. ASSP 28(1): DST-IV self-inverse property $S4^2=N\cdot I$; FFTW RODFT11 IDST-IV=(1/2N)·DST-IV; threshold 1×10^{-14}.
- Root `README.md` fixture count updated 41 → 43; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 41 → 43.
- `cargo test --workspace`: 0 FAILED, 0 ignored.

[0.12.9] — Closure XXIX

Closure XXIX — Inverse-Roundtrip Published-Reference Fixtures: NTT, STFT [patch]

Added

- Validation fixture 40 in `apollo-validation`: `ntt_inverse_roundtrip_fixture` — NTT N=4: `INTT(NTT([1,2,3,4])) = [1,2,3,4]`. Pollard (1971) Math. Proc. Cambridge Phil. Soc. 70(3): NTT inversion theorem in $\mathbb{Z}/p\mathbb{Z}$; threshold 1×10^{-12}.
- Validation fixture 41 in `apollo-validation`: `stft_hann_wola_inverse_roundtrip_fixture` — STFT frame=4,hop=2: `ISTFT(STFT([1,0,0,0])) = [1,0,0,0]`. Allen & Rabiner (1977) Proc. IEEE 65(11) WOLA synthesis; Portnoff (1980) Hann COLA; Hann w=[0,0.75,0.75,0], COLA weight=0.5625; threshold 1×10^{-12}.
- Root `README.md` fixture count updated 39 → 41; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 39 → 41.
- `cargo test --workspace`: 0 FAILED, 0 ignored.

[0.12.8] — Closure XXVIII

Closure XXVIII — Inverse-Roundtrip Published-Reference Fixtures: DHT, SFT [patch]

Added

- Validation fixture 38 in `apollo-validation`: `dht_inverse_roundtrip_fixture` — DHT N=4: `IDHT(DHT([3,-1,2,0])) = [3,-1,2,0]`. Bracewell (1983) JOSA 73(12): $H^2=NI$; inverse = (1/N)·DHT; threshold 1×10^{-14}.
- Validation fixture 39 in `apollo-validation`: `sft_inverse_roundtrip_fixture` — SFT N=4, K=1: `ISFT(SFT([1,-1,1,-1])) = [1,-1,1,-1]`. Cooley-Tukey (1965)

DFT[(−1)^n]=4·δ[k−2]; Hassanieh et al. (2012) K-sparse exact recovery; Candès & Wakin (2008) RIP; threshold 1×10^{-12}.
- Root `README.md` fixture count updated 37 → 39; two new entries appended.

Final state

- Both count assertions in `apollo-validation/suite.rs` updated 37 → 39.
- `cargo test --workspace`: 0 FAILED, 0 ignored.

[0.12.7] — Closure XXVII

Closure XXVII — Inverse-Roundtrip Published-Reference Fixtures: FWHT, QFT, SHT [patch]

Added

- Validation fixture 35 in `apollo-validation`: `fwht_inverse_roundtrip_fixture` — FWHT N=4: `IFWHT(FWHT([1,2,3,4])) = [1,2,3,4]`. Walsh (1923) Am. J. Math. 45 §2: $W_N^2 = N\cdot I$; threshold 1×10^{-14}.
- Validation fixture 36 in `apollo-validation`: `qft_inverse_roundtrip_fixture` — QFT N=4: `iqft(qft([1,0,0,0])) = [1,0,0,0]`. Shor (1994) §2: QFT_N unitary; Nielsen & Chuang (2000) §5.1; threshold 1×10^{-12}.
- Validation fixture 37 in `apollo-validation`: `sht_inverse_roundtrip_y10_fixture` — SHT lmax=1, lat=12, lon=25: dipole Y_1^0 = √(3/4π)·cosθ roundtrip; Driscoll & Healy (1994) Adv. Appl. Math. 15 Theorem 1; threshold 1×10^{-10}.
- Root `README.md` fixture count updated 34 → 37; three new entries appended.

Final state

- `cargo test --workspace`: 0 FAILED, 0 ignored, all doc-tests compile.
- Published-reference fixture count: 37.

[0.12.6] — Closure XXVI

Closure XXVI — Inverse-Roundtrip Published-Reference Fixtures: DWT, GFT, FrFT [patch]

Added

- Validation fixture 32 in `apollo-validation`: `wavelet_haar_inverse_perfect_reconstruction_fixture` — Haar DWT N=4, 1-level: `IDWT(DWT([1,-1,0,0])) = [1,-1,0,0]`. Mallat (1989) §3.1 Theorem 2 perfect reconstruction. Threshold 1×10^{-12}.
- Validation fixture 33 in `apollo-validation`: `gft_path_graph_inverse_roundtrip_fixture` — GFT K_2 path graph: `GFT⁻¹(GFT([3,-1])) = [3,-1]`. Sandryhaila & Moura (2013) ICASSP eigendecomposition invertibility. Threshold 1×10^{-12}.
- Validation fixture 34 in `apollo-validation`: `frft_inverse_roundtrip_order_half_fixture` — FrFT α=0.5, N=4: `FrFT(-0.5)(FrFT(0.5)([1,2,3,4])) = [1,2,3,4]`. Namias (1980) J.IMA 25(3)

additivity theorem $F^{-}\alpha \circ F\alpha = I$. Threshold 1×10^{-12}.

- Root `README.md` fixture count updated 31 → 34; three new fixture entries appended.

Final state

- `cargo test --workspace`: 0 FAILED, 0 ignored, all doc-tests compile.
- Published-reference fixture count: 34 (assertions in both test functions updated 31 → 34).

[0.12.5] — Closure XXV

Closure XXV — Hilbert Instantaneous Frequency + Doc/Test/PM Cleanup [patch]

Added

- `AnalyticSignal::instantaneous_frequency()` in `apollo-hilbert` (`domain/signal/analytic.rs`): computes instantaneous frequency in cycles per sample using the complex-derivative formula `f[n] = arg(conj(z[n]) · z[n+1]) / (2π)`. Returns a `Vec<f64>` of length `N − 1`. Avoids explicit phase unwrapping; values in `(−0.5, +0.5]`. Reference: Boashash (1992) Proc. IEEE 80(4).
- Validation fixture 31 in `apollo-validation`: `hilbert_instantaneous_frequency_constant_tone_fixture` — verifies that `cos(2π5·n/64)` has instantaneous frequency `5/64` at every sample (threshold 1e-10). Root `README.md` fixture count updated 30 → 31.

Added (Tests — apollo-hilbert)

- `instantaneous_frequency_constant_tone`: asserts `IF = k/N` for all N−1 samples of a single-tone cosine at `k=5`, `N=64`; tolerance 1e-10.
- `double_hilbert_negates_zero_mean_signal`: asserts `H{H{x}} = −x` for a sinusoidal input at `N=32`; tolerance 1e-10.

Changed

- `apollo-ntt-wgpu/src/verification.rs` module doc: `rust,ignore` code block converted to `rust,no_run` with `# use apollo_ntt_wgpu::NttWgpuBackend;` preamble. Eliminates the last workspace-wide ignored test; doc-test now reports "ok compile" instead of "ignored".
- `apollo-stft-wgpu/src/infrastructure/device.rs`: `execute_inverse_with_buffers` doc comment expanded from stub to full description including non-PoT delegation behaviour and `# Errors` section.
- `CHANGELOG.md`: back-filled missing Closure XXIII (0.12.3) and Closure XXIV (0.12.4) entries.
- `apollo-hilbert/README.md`: added "Instantaneous Frequency" subsection documenting formula, `N−1` length contract, and Boashash 1992 reference.

Final state

- `cargo test --workspace`: 0 FAILED, 0 ignored, all doc-tests compile.

[0.12.4] — Closure XXIV

Closure XXIV — GPU Adapter Preference + Test Runtime-Skip Conversion + Bluestein CZT Sign Fix [patch]

Changed

- All 20 `wgpu::RequestAdapterOptions::default()` sites replaced with `power_preference: PowerPreference::HighPerformance` across every wgpu crate (`apollo-fft-wgpu`, `apollo-czt-wgpu`, `apollo-mellin-wgpu`, `apollo-ntt-wgpu`, `apollo-stft-wgpu`, `apollo-radon-wgpu`, `apollo-nufft-wgpu`, `apollo-hilbert-wgpu`, `apollo-sft-wgpu`, `apollo-qft-wgpu`, `apollo-frft-wgpu`, `apollo-fwht-wgpu`, `apollo-dht-wgpu`, `apollo-sdft-wgpu`, `apollo-sht-wgpu`, `apollo-dctdst-wgpu`, `apollo-gft-wgpu`, `apollo-wavelet-wgpu`, `f16_plan.rs`, `buffer_reuse` bench). Ensures NVIDIA discrete GPU is preferred over integrated/software adapters.
- `apollo-ntt-wgpu` verification: removed all 10 `#[ignore]` attributes; converted to `let Ok(backend) = NttWgpuBackend::try_default() else { return; }` runtime-skip pattern. Tests run unconditionally on GPU-enabled hosts and skip silently on CI.
- `apollo-stft-wgpu` verification: removed all 7 `#[ignore]` attributes; same runtime-skip pattern applied.
- `apollo-stft-wgpu/src/infrastructure/shaders/stft_chirp.wgsl`: corrected all four Bluestein sign errors (`premul_fwd`: `exp(−πi·n²/N)`, `premul_inv`: `exp(+πi·n²/N)`, `postmul_fwd`: `exp(−πi·k²/N)`, `postmul_inv`: `exp(+πi·n²/N)/N`); added new `stft_chirp_pointmul_fwd` entry point that conjugates the stored kernel `h_stored = exp(−πi·j²/N)` to recover `h_fwd = exp(+πi·j²/N)`.
- `StftChirpData` (`chirp.rs`): added `pointmul_fwd_pipeline: wgpu::ComputePipeline` field; `new()` builds pipeline from `stft_chirp_pointmul_fwd` entry point.
- `kernel.rs` forward CZT dispatch (Pass C): uses `pointmul_fwd_pipeline` instead of `pointmul_pipeline`; inverse Pass C unchanged.
- `device.rs`: added non-PoT guards to `execute_forward_with_buffers` and `execute_inverse_with_buffers` that delegate to the allocating Chirp-Z path and copy results into `fwd_output_host` / `inv_output_host`.
- `stft-wgpu` forward CZT test tolerance updated `1e-2 → 2e-2` (analytically justified by f32 GPU argument-reduction at phase magnitudes up to ~1254 rad for N=400).

Final state

- `cargo test --workspace`: 0 FAILED, 0 ignored across all 38+ crates and all doc-tests.

[0.12.3] — Closure XXIII

Closure XXIII — ARCHITECTURE.md Capability Annotations + Validation Fixtures 29–30 [patch]

Changed

- `ARCHITECTURE.md` Mixed-Precision Capability Table: added `"forward + inverse CZT"` and `"forward + inverse Mellin spectrum"` annotations to the `Notes` column for `apollo-czt-wgpu` and `apollo-mellin-wgpu`, matching the bidirectional-WGPU annotation pattern already established for other transform pairs.

Added

- `apollo-validation`: two new published-reference fixtures (fixtures 29 and 30).
  - `czt_inverse_vandermonde_roundtrip_fixture`: N=4 Björck-Pereyra Vandermonde solve, threshold 1e-12. Validates exact numeric contract from Björck & Pereyra (1970).
  - `mellin_inverse_spectrum_constant_roundtrip_fixture`: N=32 constant signal IDFT + exp-resample roundtrip, threshold 1e-10.
  - `published_real_fixture_with_threshold` helper function added.
  - README fixture count updated 28 → 30.
  - `validation_suite_produces_value_semantic_reports` assertion updated to 30.
  - All 30 fixtures pass.

[0.12.2] — Closure XXII

Closure XXII — GPU Benchmark Runner Workflow + Root README Correction [patch]

Added

- `.github/workflows/gpu-benchmarks.yml`: manual `workflow_dispatch` workflow targeting a labeled self-hosted runner (`[self-hosted, gpu, apollo]`) to execute the WGPU Criterion benchmark suites and upload the output bundle as a workflow artifact.
- `scripts/run_gpu_benchmarks.ps1`: reproducible GPU benchmark runner script that stages logs, `manifest.json`, `summary.md`, and `target/criterion` under `.benchmarks/gpu-runner/`.
- `.benchmarks/gpu-runner/.gitkeep`: tracked anchor for benchmark artifact staging.

Changed

- Root `README.md`: corrected stale CZT/Mellin/STFT/Radon WGPU capability prose and added a dedicated "GPU Benchmark Runner" section documenting labels, workflow entry point, executed benches, and output staging path.
- `.gitignore`: ignores generated `.benchmarks/gpu-runner/*` outputs while keeping the tracked directory anchor.

[0.12.1] — Closure XXI

Closure XXI — README Documentation Sync for v0.2.0 Inverse Additions [patch]

Changed

- `apollo-czt/README.md`: added "Inverse Transform" section documenting `CztPlan::inverse`, Björck-Pereyra Vandermonde solve, and `CztError::NotInvertible` conditions (non-square plan, node collision). Updated "Verification" section to list inverse roundtrip and rejection tests.
- `apollo-mellin/README.md`: added "Inverse Transform" section documenting `MellinPlan::inverse_spectrum`, IDFT + exp-resample algorithm, and `MellinError::SpectrumLengthMismatch`. Updated "Verification" section.
- `apollo-czt-wgpu/README.md`: updated "Execution Contract" to reflect forward+inverse support; added adjoint formula description; removed stale "inverse unsupported" prose. Updated "Verification" section to mention GPU inverse roundtrip test.
- `apollo-mellin-wgpu/README.md`: updated "Execution Contract" to document two-pass GPU inverse (IDFT + exp-resample); updated capability and verification prose.
- `checklist.md`: added completed Closure XX entry (22 checklist items).

[0.12.0] — Closure XX

Closure XX — CPU + GPU Inverse Transforms: CZT and Mellin [minor]

Added — apollo-czt v0.2.0

- `CztPlan::inverse(spectrum)` — exact Vandermonde solve via Björck-Pereyra algorithm (`O(N²)` in-place Newton evaluation). Returns `CztError::NotInvertible` when `M ≠ N` or when Vandermonde nodes collide (denominator below `f64::EPSILON * 1024`).
- `CztStorage::inverse_into` — default method adapting `inverse` to in-place storage API.
- `CztError::NotInvertible { reason: &'static str }` variant.
- 5 value-semantic tests: roundtrip at DFT parameters, general `A` offset, non-unit `W` spacing, rejection of rectangular plans, rejection of wrong spectrum length.

Added — apollo-mellin v0.2.0

- `MellinPlan::inverse_spectrum(spectrum, out_min, out_max, output)` — IDFT of log-domain spectrum then exp-resample from log-grid to linear domain. Rayon-parallel IDFT for `N ≥ 256`.
- `inverse_log_frequency_spectrum` and `exp_resample` exported from `lib.rs`.
- `MellinError::SpectrumLengthMismatch` variant.
- 4 value-semantic tests: constant-signal roundtrip (`ε < 1e-10`), linear-signal roundtrip (interpolation error `< 0.1` for `N = 64`), wrong-length rejection, invalid-bounds rejection.

Added — apollo-czt-wgpu v0.2.0

- `czt_inverse` WGSL entry point: adjoint formula `x[n] = (A^n/N)·∑_k X[k]·W^{-nk}`, exact for unitary DFT parameters.
- `MellinGpuKernel::inverse_pipeline` field; `execute_inverse` dispatches over `N` threads.

- `CztWgpuBackend::execute_inverse(plan, spectrum)` — validates `M == N`, delegates to kernel.
- `WgpuCapabilities::forward_inverse` constructor.
- 2 new GPU tests: roundtrip at DFT parameters, rejection of non-square plan.

Added — apollo-mellin-wgpu v0.2.0

- `mellin_inverse_spectrum` WGSL kernel: IDFT pass, spectrum → log-domain samples.
- `mellin_exp_resample` WGSL kernel: exp-resample pass, log-domain → linear output.
- `InverseMellinParamsPod` uniform struct (32 bytes, reuses params buffer slot).
- `MellinGpuKernel::inverse_spectrum_pipeline`, `exp_resample_pipeline`, `inv_params_buffer` fields; `execute_inverse` dispatches two GPU passes + readback.
- `MellinWgpuBackend::execute_inverse(plan, spectrum, out_min, out_max, out_len)`.
- `WgpuCapabilities::forward_inverse` constructor.
- 2 new GPU tests: constant-signal roundtrip (`ε < 5e-4`), invalid-domain rejection.

[0.10.0] — Closure XIX

Closure XIX — StftGpuBuffers Non-PoT Scratch Sizing [minor]

Changed

- `StftGpuBuffers::new` now accepts arbitrary `frame_len` (not just power-of-two).
  Scratch buffers (`re_scratch_buf`, `im_scratch_buf`, `frame_data_buf`) are automatically sized to `frame_count × M` where `M = chirp_padded_len(frame_len)` when `!frame_len.is_power_of_two()`,
  and `frame_count × frame_len` when PoT.
- `StftWgpuBackend::make_buffers`, `execute_forward_with_buffers`, `execute_inverse_with_buffers`
  no longer return `WgpuError::FrameLenNotPowerOfTwo`; non-PoT `frame_len` now dispatches via the buffer-reuse path (kernel auto-selects Chirp-Z for non-PoT at dispatch time).
- Module docstring in `buffers.rs` updated: PoT constraint removed; Closure XIX note added.

Added (Tests)

- `make_buffers_accepts_non_power_of_two_frame_len_structurally`: structural verification that `make_buffers` accepts non-PoT without returning `FrameLenNotPowerOfTwo`.
- `forward_buffers_non_pot_frame_len_400_when_device_exists` (GPU-gated): buffer-reuse forward at `frame_len=400`, GPU output matched against CPU `apollo-stft` reference.
- `inverse_buffers_non_pot_frame_len_400_when_device_exists` (GPU-gated): buffer-reuse inverse at `frame_len=400`, WOLA interior-sample recovery comparison.

Removed

- Panic condition `assert!(frame_len.is_power_of_two())` in `StftGpuBuffers::new`.

[0.9.0] — Closure XVIII

Closure XVIII — Non-Power-of-Two STFT GPU Path (Bluestein/Chirp-Z) [minor]

Added

- `crates/apollo-stft-wgpu/src/infrastructure/shaders/stft_chirp.wgsl`: Five-pass WGSL compute shader implementing the Bluestein Chirp-Z mapping for the STFT: `premul_fwd`, `premul_inv`, `pointmul`, `postmul_fwd`, `postmul_inv`. Hann analysis/synthesis windows and $\exp(\pm\pi i \cdot n^2/N)$ chirp twiddles are applied on-GPU.
- `crates/apollo-stft-wgpu/src/infrastructure/shaders/stft_chirp_fft.wgsl`: Radix-2 sub-FFT shader operating on chirp working buffers: `chirp_fft_bitrev`, `chirp_fft_butterfly_fwd`, `chirp_fft_butterfly_inv`, `chirp_fft_scale`.
- `crates/apollo-stft-wgpu/src/infrastructure/chirp.rs`: `StftChirpData` struct — pre-allocated GPU resources (chirp kernel H, working buffers, bind groups, pipelines) for the Bluestein path. `chirp_padded_len(n)` returns `(2n-1).next_power_of_two()`.
- `design_history_file/adr_stft_wgpu_non_pot_chirpz.md`: ADR for the Chirp-Z dispatch strategy (Bluestein 1970, Rabiner 1969).
- `ndarray = "0.16"` added as a regular (non-dev) dependency to `apollo-stft-wgpu` for CPU-side chirp kernel construction in `StftChirpData::new`.

Changed

- `execute_forward` and `execute_inverse` in `StftWgpuBackend` (device.rs) no longer return `WgpuError::FrameLenNotPowerOfTwo`; non-power-of-two `frame_len` now automatically dispatches the Bluestein/Chirp-Z path.
- `kernel.rs`: non-PoT `frame_len` delegates to `execute_forward_fft_chirp` / `execute_inverse_chirp` instead of returning an error.
- `WgpuError::FrameLenNotPowerOfTwo` doc comment updated to reflect that this variant is no longer returned by primary dispatch; it may still be returned by `make_buffers`, `execute_forward_with_buffers`, `execute_inverse_with_buffers`.
- Verification tests `forward_rejects_non_power_of_two_frame_len` and `inverse_rejects_non_power_of_two_frame_len` renamed and inverted to `forward_accepts_non_power_of_two_frame_len_chirpz` and `inverse_accepts_non_power_of_two_frame_len_chirpz`.

Added (Tests)

- `forward_accepts_non_power_of_two_frame_len_structurally`: structural check that non-PoT no longer returns `FrameLenNotPowerOfTwo`.
- `forward_accepts_non_power_of_two_frame_len_chirpz`: same for `execute_forward`.
- `inverse_accepts_non_power_of_two_frame_len_chirpz`: same for `execute_inverse`.
- `forward_chirpz_non_pot_frame_len_400_when_device_exists` (GPU-gated): GPU forward Chirp-Z at frame_len=400, compared to CPU `apollo-stft` reference, TOL=1e-2.
- `inverse_chirpz_non_pot_frame_len_400_when_device_exists` (GPU-gated): GPU inverse Chirp-Z at frame_len=400, WOLA interior-sample recovery, TOL=5e-2.

[0.8.5] — Closure XVII

Closure XVII — STFT GPU Buffer-Reuse Criterion Benchmarks + README Usage Documentation [patch]

Added

- `bench_forward_reuse` benchmark group in `crates/apollo-stft-wgpu/benches/stft_bench.rs`:
  head-to-head comparison of `execute_forward` (allocating) vs `execute_forward_with_buffers` (buffer-reuse) at `frame_len` ∈ {256, 512, 1024}. Pre-allocates `StftGpuBuffers` outside the bench loop; measures only signal upload + GPU dispatch + readback per iteration.
- `bench_inverse_reuse` benchmark group: same head-to-head comparison for `execute_inverse` vs `execute_inverse_with_buffers`.
- Both groups added to `criterion_group!(benches, …)` in `stft_bench.rs`.
- Updated module docstring in `stft_bench.rs` to describe both allocating and buffer-reuse paths and their mathematical basis.
- "Buffer Reuse" section in `crates/apollo-stft-wgpu/README.md`: usage snippet showing `make_buffers` → `execute_forward_with_buffers` → `fwd_output()` pattern, constraint notes for `FrameLenNotPowerOfTwo` and `LengthMismatch` errors.
- "Benchmarks" section in `README.md`: table of all four benchmark groups with description and `cargo bench -p apollo-stft-wgpu` invocation.

[0.8.4] — Closure XVI

Closure XVI — StftGpuBuffers Pre-allocated Buffer Reuse [minor]

Added

- `StftGpuBuffers` struct in `crates/apollo-stft-wgpu/src/infrastructure/buffers.rs`:
  pre-allocates all GPU data buffers, staging buffers, bind groups, and per-stage butterfly uniform buffers for a fixed `(frame_count, frame_len, signal_len, hop_len)` quad. Eliminates 5–8 `device.create_buffer` calls, 4+ `device.create_bind_group` calls, and `log₂(N)` uniform-buffer allocations per dispatch.
- `StftWgpuBackend::make_buffers(plan, signal_len)`: constructs a `StftGpuBuffers` for a given plan shape with validation identical to the allocating paths.
- `StftWgpuBackend::execute_forward_with_buffers(plan, signal, buffers)`: zero-allocation
  forward STFT dispatch; result in `buffers.fwd_output()`.
- `StftWgpuBackend::execute_inverse_with_buffers(plan, spectrum, signal_len, buffers)`:
  zero-allocation inverse STFT dispatch; result in `buffers.inv_output()`.
- `StftGpuKernel::execute_forward_fft_with_buffers` and `execute_inverse_with_buffers`:
  kernel-level buffered dispatch methods.
- Verification test
  `reusable_buffers_match_allocating_forward_and_inverse_when_device_exists`
  (#[ignore = "requires wgpu device"]): asserts bit-exact agreement between allocating and

buffered paths on a bin-aligned sinusoid (k=16, frame_len=512);
verifies idempotent second-call buffer reuse.

- `pub use infrastructure::buffers::StftGpuBuffers` re-exported from crate root.

[0.8.3] — Closure XV

Closure XV — Radon FBP GPU Criterion Benchmarks

Added

- `crates/apollo-radon-wgpu/benches/radon_wgpu_bench.rs`: criterion benchmark suite with `radon_wgpu_forward` and `radon_wgpu_fbp` groups.
- `criterion = "0.5"` in `apollo-radon-wgpu` dev-dependencies.
- `[[bench]] name = "radon_wgpu_bench" harness = false` in `apollo-radon-wgpu/Cargo.toml`.

[0.8.2] — Closure XIV

Closure XIV — Dead-Code Removal: $O(N^2)$ Forward Pipeline

Removed

- `StftGpuKernel::execute()`: $O(N^2)$ direct DFT forward method (superseded by Closure XII FFT path).
- `forward_pipeline` field and shader creation code from `StftGpuKernel::new()`.
- `shaders/stft.wgsl`: $O(N^2)$ forward DFT WGSL shader (superseded by `stft_forward_fft.wgsl`).
- `stft_inverse_frames` entry point from `stft_inverse.wgsl` (superseded by Closure XI FFT inverse).

Changed

- `stft_inverse.wgsl` file header updated to reflect single-pass OLA role.
- `kernel.rs` module docstring, `WORKGROUP_SIZE` comment, struct docs updated.

[0.8.1] — Closure XIII

Closure XIII — STFT GPU Criterion Benchmarks

Added

- `crates/apollo-stft-wgpu/benches/stft_bench.rs`: criterion benchmark suite with `bench_forward_fft` and `bench_inverse_fft` groups.
- `criterion = { version = "0.5", features = ["html_reports"] }` in `apollo-stft-wgpu` dev-dependencies.
- `[[bench]] name = "stft_bench" harness = false` in `apollo-stft-wgpu/Cargo.toml`.

[0.8.0] — Closure XII

Closure XII — STFT Forward-Path GPU FFT Acceleration

Added

- `stft_forward_fft.wgsl`: new GPU shader with `stft_fwd_pack_window`, `stft_fwd_bitrev`, `stft_fwd_butterfly`, `stft_fwd_interleave` entry points (DFT twiddle `exp(−2πi·k/N)`).
- `FwdFftStageParams` struct (16 bytes, fields: frame_count, frame_len, hop_len, stage).
- `StftGpuKernel::execute_forward_fft`: O(N log N) GPU forward STFT, PoT frame_len required.
- `FrameLenNotPowerOfTwo` guard in `StftWgpuBackend::execute_forward`.
- Tests: `forward_rejects_non_power_of_two_frame_len`, `forward_fft_roundtrip_large_frame_when_device_exists`.

Changed

- `StftWgpuBackend::execute_forward` now routes to the FFT-accelerated path and requires power-of-two `frame_len` (previously accepted any `frame_len` via $O(N^2)$ direct DFT).

Breaking

- `execute_forward` with non-power-of-two `frame_len` now returns `Err(WgpuError::FrameLenNotPowerOfTwo)` instead of computing a result.

[0.7.0] — Closure XI

Added

- `apollo-stft-wgpu`: GPU STFT inverse O(N log N) acceleration. New `stft_inverse_fft.wgsl` with four entry points implementing a batched Cooley-Tukey Radix-2 DIT IFFT: `stft_deinterleave` (interleaved complex f32 → split re/im scratch), `stft_bitrev` (bit-reversal permutation, batched), `stft_butterfly` (one Radix-2 DIT stage per dispatch; IDFT twiddle exp(+2πi·k/N)), `stft_scale_and_window` (1/N scale + Hann synthesis window → frame_data). Two-bind-group architecture: group 0 = 4 data bindings (shared), group 1 = per-stage `FftStageParams` uniform (pre-allocated, one per butterfly pass). All passes in one `CommandEncoder`; implicit per-pass memory barriers ensure write visibility. OLA pass unchanged. Replaces the $O(N^2)$ `stft_inverse_frames` pipeline. Formal basis: Cooley & Tukey (1965); Allen & Rabiner (1977) Theorem 1. [minor]
- `apollo-stft-wgpu`: `WgpuError::FrameLenNotPowerOfTwo { frame_len: usize }` error variant. Returned by `execute_inverse` when `frame_len` is not a power of two (Radix-2 invariant); checked before GPU buffer allocation in `device.rs` and at IFFT entry in `kernel.rs`. [minor]
- `apollo-stft-wgpu`: `inverse_rejects_non_power_of_two_frame_len` test (CPU-only; asserts `FrameLenNotPowerOfTwo { frame_len: 6 }` for frame_len=6). [patch]
- `apollo-stft-wgpu`: `inverse_roundtrip_large_frame_1024_samples_when_device_exists` GPU-gated test (frame_len=1024, $\log_2 N=10$ butterfly stages, hop=512, signal_len=8192, analytic sine reference, TOL=5e-3). [patch]

[0.6.0] — Closure X

Added

- `apollo-radon-wgpu`: GPU ramp-filtered backprojection (FBP). New `radon_fbp_filter.wgsl` entry point `radon_fbp_filter` applies the Ram-Lak ramp filter to each sinogram projection row via circular convolution with the impulse response `h = IFFT(R)`, `R[k] = 2π·|signed_k|/(N·Δ)` (Bracewell & Riddle 1967; Shepp & Logan 1974). Filter kernel computed host-side from `apollo_radon::ramp_filter_projection([1,0,...], Δ)` and cast to f32. Two-pass single `CommandEncoder` (filter → backproject). Host-side `π/angle_count` normalization. `RadonWgpuBackend::execute_filtered_backproject`. `supports_filtered_backprojection: bool` field added to `WgpuCapabilities`. `WgpuCapabilities::forward_inverse_and_fbp` constructor added. [minor]
- `apollo-radon-wgpu`: `backproject_satisfies_adjoint_identity_when_device_exists` test verifies the Radon adjoint identity ⟨A·f, g⟩_sinogram = ⟨f, A†·g⟩_image (Natterer 2001, §II.2) to relative tolerance 5e-3. [patch]
- `apollo-stft-wgpu`: `inverse_roundtrip_for_multiple_cola_parameter_sets` tests three COLA-compliant parameter sets (frame_len=8/hop=4, 16/8, 32/16) with analytical reference signals. TOL=5e-3. [patch]

Fixed

- `README.md`: stale WGPU crate descriptions for `apollo-radon-wgpu` (was "forward only"), `apollo-stft-wgpu` (was "forward only"), `apollo-hilbert-wgpu` (was "inverse unsupported"), and `apollo-sdft-wgpu` (was "inverse unsupported"). All now accurately describe implemented GPU inverse capabilities. [patch]
- `ARCHITECTURE.md`: Mixed-Precision Capability Table notes for `apollo-radon-wgpu`, `apollo-stft-wgpu`, `apollo-hilbert-wgpu`, and `apollo-sdft-wgpu` updated to reflect inverse capability status. [patch]

[0.5.0] — Closure IX

Added

- `apollo-stft-wgpu`: GPU inverse STFT via two-pass Weighted Overlap-Add (WOLA). New WGSL file `stft_inverse.wgsl` with entry points `stft_inverse_frames` (per-(frame, local_j) windowed IDFT: `frame_data[m·N+j] = (1/N)·Re{Σ_k X[m,k]·exp(+2πi·k·j/N)}·hann(j)`, spectrum as interleaved f32 pairs) and `stft_inverse_ola` (per-sample `y[n] = Σ_m frame_data[m·N+(n−start_m)] / Σ_m hann(n−start_m)²`, `start_m = m·hop−N/2`). Both passes share the existing 3-binding layout in one `CommandEncoder`. `StftGpuKernel::execute_inverse` (2-pass single encoder). `StftWgpuBackend::execute_inverse(plan, spectrum, signal_len)` + `execute_inverse_typed_into`. `WgpuCapabilities::forward_and_inverse` constructor added. Basis: WOLA identity (Allen–Rabiner 1977, Theorem 1). [minor]
- `apollo-radon-wgpu`: GPU Radon adjoint backprojection. New WGSL file `radon_backproject.wgsl` with entry `radon_backproject`: per-pixel `bp[r,c] = Σ_θ interp(sinogram[θ,·], x·cosθ + y·sinθ)` with linear interpolation and out-of-range zero-

clamping, reusing the forward bind group layout. `RadonGpuKernel::execute_backproject`. `RadonWgpuBackend::execute_inverse(plan, sinogram, angles)` + `execute_inverse_flat_typed`. `WgpuCapabilities::forward_and_inverse` constructor added. `SinogramShapeMismatch` error variant added. Basis: Radon adjoint operator (Natterer 2001, §II.2). [minor]

Fixed

- `gap_audit.md`: open-gap note incorrectly stated "CPU inverse paths are implemented" for `apollo-czt-wgpu` and `apollo-mellin-wgpu`. Corrected: those crates have no CPU inverse defined; `execute_inverse` returns `UnsupportedExecution` by architectural design, not by deferral. [patch]

[0.4.0] — Closure VIII

Added

- `apollo-hilbert-wgpu`: GPU inverse Hilbert transform. New WGSL entry point `hilbert_inverse_mask` recovers the original real-signal DFT spectrum from the DFT of the quadrature signal: positive bins X[k]=j·Q[k], negative bins X[k]=-j·Q[k], DC and Nyquist zeroed (unrecoverable; Bracewell 1965). New `HilbertGpuKernel::execute_inverse` runs 3 sequential passes in one command encoder (DFT of quadrature → inverse mask → IDFT of recovered spectrum). Exposed via `HilbertWgpuBackend::execute_inverse` and `execute_inverse_typed_into`. `WgpuCapabilities::forward_and_inverse` constructor added. [minor]
- `apollo-sdft-wgpu`: GPU inverse SDFT. New WGSL entry point `sdft_inverse_bins` computes x[n]=(1/K)·Σ_{b=0}^{K-1} X[b]·exp(+2πi·b·n/K). Complex bins passed as interleaved f32 pairs. Separate `forward_pipeline` and `inverse_pipeline` in `SdftGpuKernel`. Exposed via `SdftWgpuBackend::execute_inverse` and `execute_inverse_typed_into`. `WgpuCapabilities::forward_and_inverse` constructor added. [minor]

Fixed

- `apollo-hilbert-wgpu`: pre-existing bug in `hilbert_inverse_dft` WGSL: real accumulator was written back as a stale self-assign (`inout_b[n].re = original`); corrected to `inout_b[n].re = acc.x * scale`. [patch]
- `apollo-czt`: proptest `bluestein_equals_direct_for_arbitrary_parameters` used a fixed 1e-9 absolute tolerance, which is violated when `|w|>1` amplifies output magnitudes by up to |w|^((N-1)^2/2). Tolerance changed to `1e-9·max(|direct[k]|,1.0)` (relative bound). Formal basis: Bluestein relative error ≤ C·$\log_2$(p)·ε_machine ≈12·2.2e-16≈2.6e-15 (Higham §3.10). [patch]

[0.3.0] — Closure VII

Added

- Six new published-reference fixtures in `apollo-validation`: SFT 1-sparse alternating tone, SHT monopole Y_0^0 coefficient, STFT rectangular-window impulse frame, Hilbert cosine-to-sine 4-point,

Mellin constant-function first moment, Radon θ=0 column-impulse projection. Fixture count rises from 22 to 28. [minor]

- Proptest coverage for four CPU transform crates previously lacking property tests: `apollo-czt` (Bluestein-vs-direct parity, spiral-collapse to DFT, linearity), `apollo-frft` (roundtrip, additivity, linearity), `apollo-nufft` (DC-mode invariant, fast-path tracks exact, Type-1 linearity), `apollo-sft` (K-sparse exact recovery, Parseval top-K, retained bins equal DFT). [minor]

Changed

- `apollo-frft-wgpu`: `UnitaryFrftGpuKernel::execute` refactored from 3 separate command encoder submissions to a single command encoder with 3 sequential compute passes followed by a copy command. This reduces CPU–GPU round-trips from 4 submits + 5 polls to 1 submit + 2 polls while preserving the cross-pass write-visibility guarantee via the implicit memory barrier at each `ComputePass` boundary (WebGPU spec §3.4 sequential pass ordering). [patch]
- `design_history_file/`: removed stale shadow copies of `backlog.md`, `checklist.md`, and `gap_audit.md` (root artifacts are the SSOT). `adr_unitary_frft.md` retained as the authoritative ADR. [patch]

Fixed

- `README.md`: updated `apollo-validation` fixture count from 10 (stale) to 28 (final Closure VII count) and replaced the stale fixture list with the complete 28-fixture inventory. [patch]

[0.2.0] — Closure VI (NTT WGPU O(N log N), workspace unblock, expanded fixtures)

Added

- `apollo-ntt-wgpu`: O(N log N) Cooley-Tukey DIT butterfly shader (`ntt_butterfly` + `ntt_scale` entry points), replacing the $O(N^2)$ DFT loop. $\text{Log}_2(N)$ butterfly passes plus optional scale pass encoded in one command buffer with dynamic uniform offsets. [major]
- Two published-reference fixtures: NTT N=16 impulse (Pollard 1971) and NTT N=16 polynomial product via convolution theorem. Fixture count 20 → 22. [minor]
- CPU-only proptest tests in `apollo-ntt-wgpu` verification: `cpu_roundtrip_preserves_residue_class` and `convolution_theorem_holds_for_arbitrary_pairs`. [minor]

Changed

- `apollo-ntt-wgpu`: removed `apollo_fft::PrecisionProfile` cross-domain import and `default_precision_profile` field from capabilities; NTT is exact integer arithmetic. [minor]
- GPU-dependent tests in `apollo-ntt-wgpu` annotated `#[ignore = "requires wgpu device"]` replacing silent early-return skips. [patch]

Fixed

- Workspace compilation regression: reverted `apollo-fft/Cargo.toml` package name from `"apollo"` to `"apollo-fft"` and corrected dependent crate path keys. [patch]
- Removed `#![allow(unused_imports)]` from `apollo-ntt/src/lib.rs` and unused `Array1` import. [patch]

[0.1.9] — Closure V (GPU Unitary FrFT, ADR, published fixtures)

Added

- `apollo-frft-wgpu`: `UnitaryFrftGpuKernel` implementing DFrFT_a(x)=V·diag(exp(−iakπ/2))·V^T·x on GPU via three-submission pattern. `FrftWgpuBackend` exposes `plan_unitary`, `execute_unitary_forward`, `execute_unitary_inverse`. [minor]
- Three published-reference fixtures (count 17 → 20): FrFT unitary order-2 reversal (Candan 2000), DWT Haar one-level detail (Haar 1910/Mallat 1989), SDFT bin-zero unit impulse. [minor]
- `design_history_file/adr_unitary_frft.md`: ADR documenting Grünbaum eigendecomposition algorithm selection, unitarity proof, GPU ordering guarantee, and tolerance derivation. [patch]
- `ARCHITECTURE.md`: "Key: Unitary FrFT" subsection with CPU/GPU plan comparison table. [patch]

[0.1.8] — Closure IV (FrFT unitarity, DCT-I/IV/DST-I/IV WGPU)

Added

- `apollo-frft`: `GrunbaumBasis` and `UnitaryFrftPlan` (Candan 2000 eigendecomposition). $O(N^3)$ construction, $O(N^2)$ per call, provably unitary for all real orders. [minor]
- `apollo-dctdst-wgpu`: WGSL shader modes for DCT-I (mode 4), DCT-IV (mode 5), DST-I (mode 6), DST-IV (mode 7) with correct self-inverse scales. [minor]

[0.1.7] — Closure III (validation mock removal, published fixtures, DCT-I/IV/DST-I/IV CPU)

Added

- 7 published-reference fixtures (count 10 → 17): FFT inverse, DCT-II inverse pair, DHT self-reciprocal, FWHT 2-point, QFT 2-point, CZT spiral-collapse, GFT path graph. [minor]
- `apollo-validation` GPU suite: real 4×4×4 GpuFft3d roundtrip replacing hardcoded `passed: true` stub. [major]

Fixed

- `apollo-validation` precision profile forward errors computed from actual GPU vs CPU f64 reference comparison. [patch]

[0.1.6] — Closure II (fixture expansion, capability table)

Added

- Published-reference fixtures expanded to 10. [minor]
- `ARCHITECTURE.md` Mixed-Precision Capability Table (authoritative per-crate precision record). [patch]

[0.1.5] — Performance & Native GPU Precision

Added

- `apollo-fft-wgpu`: `GpuFft3dF16Native` behind `native-f16` feature; native f16 arithmetic with `enable f16` WGSL. Bluestein chirp-Z f16 shader for non-power-of-two sizes. [minor]
- Criterion buffer-reuse benchmarks for `apollo-nufft-wgpu` and `apollo-fft-wgpu`. [minor]
- NUFFT and FFT reusable-buffer `with_buffers` façade methods. [minor]

[0.1.4] — Extension Phase (mixed precision rollout, typed storage)

Added

- Mixed-precision typed storage APIs across all CPU and WGPU transform crates. [minor]
- Exact quantized `u32` NTT-WGPU residue storage and reusable buffer dispatch. [minor]

[0.1.3] — GPU Numerical Kernels (NUFFT, SHT, SFT, STFT, Wavelet, DCT/DST)

Added

- GPU fast NUFFT 1D/3D Kaiser-Bessel gridding paths. [minor]
- WGPU backends for SHT, STFT, Haar DWT, DCT-II/III/DST-II/DST-III. [minor]

[0.1.2] — Core Transform Crates (GFT, QFT, SDFT, SFT, Radon, Mellin, Hilbert, Wavelet, STFT, CZT, FWHT)

Added

- New CPU transform crates with WGPU backends. [minor]

[0.1.1] — Foundation (FFT, DHT, DCT/DST, NTT, NUFFT)

Added

- Core CPU transform crates with O(N log N) kernels. [minor]
- `apollo-validation` published-reference suite with 10 initial fixtures. [minor]

[0.1.0] — Initial release

Added

- Workspace skeleton: `apollo-fft`, `apollo-fft-wgpu`, `apollo-nufft`, `apollo-nufft-wgpu`, `apollo-validation`, `apollo-python`. [minor]